The Abstract Factory is a creational design pattern that allows you to produce families of related objects without specifying their concrete classes. It acts as a "factory of factories."

Standard Analogy: Furniture Factory
Imagine a furniture shop that produces products in specific styles: Modern and Victorian. A Modern chair must match a Modern sofa. The Abstract Factory ensures that when you order a "Furniture Set," all individual items (Chair, Sofa) belong to the same style family.

```
// Abstract Products
interface Chair {
    void sitOn();
}

interface Sofa {
    void lieOn();
}

// Concrete Products: Modern Family
class ModernChair implements Chair {
    public void sitOn() { System.out.println("Sitting on a sleek modern chair.")
}

class ModernSofa implements Sofa {
    public void lieOn() { System.out.println("Lying on a minimalist modern sofa.'
}

// Concrete Products: Victorian Family
class VictorianChair implements Chair {
    public void sitOn() { System.out.println("Sitting on a royal Victorian chair
}

class VictorianSofa implements Sofa {
    public void lieOn() { System.out.println("Lying on a heavy Victorian sofa.")
}
```

```java
// Abstract Factory
interface FurnitureFactory {
    Chair createChair();
    Sofa createSofa();
}

// Concrete Factories
class ModernFurnitureFactory implements FurnitureFactory {
    public Chair createChair() { return new ModernChair(); }
    public Sofa createSofa() { return new ModernSofa(); }
}

class VictorianFurnitureFactory implements FurnitureFactory {
    public Chair createChair() { return new VictorianChair(); }
    public Sofa createSofa() { return new VictorianSofa(); }
}

// Test Class
public class AbstractFactoryTest {
    public static void main(String[] args) {
        // Client decides to use the Modern style
        FurnitureFactory factory = new ModernFurnitureFactory();

        Chair chair = factory.createChair();
        Sofa sofa = factory.createSofa();
```

Why this is an Abstract Factory Pattern

- **Family of Products:** It manages multiple related products (`Chair` and `Sofa`) rather than just one.
- **Consistency:** It guarantees that products from one factory are compatible (e.g., you won't accidentally mix a Victorian chair with a Modern sofa).
- **High-Level Abstraction:** The client code (`AbstractFactoryTest`) works with the abstract interfaces (`FurnitureFactory` , `Chair` , `Sofa`) and is entirely independent of the concrete classes (`ModernChair` , etc.).
- **Open/Closed Principle:** You can introduce new variants of furniture families (e.g., ArtDeco) without breaking existing client code.

Core Differences

- **Scale and Scope:** The **Factory Method** is designed to produce a **single product type** [1] [2] . In contrast, the **Abstract Factory** is a "factory of factories" used to create **families of related or dependent objects** (e.g., a chair and a sofa in the same style) [1] [...] .
- **Implementation Mechanism:** The **Factory Method relies on inheritance** [1] . It defines an interface or abstract class for creating an object but lets subclasses decide which specific concrete class to instantiate [2] [5] . The **Abstract Factory uses composition** [1] [6] . A client is provided with a concrete factory object, and it calls that object's methods to get product instances [7] [8] .
- **Abstraction Level:** Abstract Factory is considered a **higher layer of abstraction** over the Factory Method [9] [10] . It often uses Factory Methods within its own implementation to create the individual products that make up a family [11] .
- **Complexity:** The Abstract Factory is generally **more complex** because it requires defining multiple interfaces and concrete classes for both the factories and the various products they create [1] [...] .